

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

CHETAN BARAKKI SATISH KUMAR (1BM24CS081)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by CHETAN BARAKKI SATISH KUMAR(1BM24CS081), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	1
2	Implement Johnson Trotter algorithm to generate permutations.	3
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	6
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	8
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	10
6	Implement 0/1 Knapsack problem using dynamic programming	12
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	14
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	16
9	Implement Fractional Knapsack using Greedy technique.	20
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	22
11	Implement "N-Queens Problem" using Backtracking.	24
12	Leetcode problems	26

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int adj[100][100];
int stack[100], visited[100] = {0};
int top = -1;
int n;

void dfs(int v)
{
    visited[v] = 1;

    for (int i = 0; i < n; i++)
    {
        if (adj[v][i] && !visited[i])
        {
            dfs(i);
        }
    }

    stack[++top] = v;
}

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &adj[i][j]);
        }
    }

    for (int i = 0; i < n; i++)
    {
        if (!visited[i])
        {
            dfs(i);
        }
    }

    printf("Topological Order: ");
    for (int i = top; i >= 0; i--)
    {
        printf("%d ", stack[i]);
    }

    return 0;
}
```

```
}  
Enter number of vertices: 2  
Enter adjacency matrix:  
0 1  
1 0  
Topological Order: 0 1  
  
...Program finished with exit code 0  
Press ENTER to exit console.[]
```

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

int a[10], d[10], n;

int mobile()
{
    int i, m = 0, p = -1;

    for (i = 0; i < n; i++)
    {
        if (d[i] == -1 && i - 1 >= 0 && a[i] > a[i - 1])
        {
            if (a[i] > m)
            {
                m = a[i];
                p = i;
            }
        }
        else if (d[i] == 1 && i + 1 < n && a[i] > a[i + 1])
        {
            if (a[i] > m)
            {
                m = a[i];
                p = i;
            }
        }
    }
}

return p;
}

int main()
{
    int i, p, t, v;

    printf("Enter n: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        a[i] = i + 1;
        d[i] = -1;
    }

    while (1)
    {
        for (i = 0; i < n; i++)
            printf("%d ", a[i]);

        printf("\n");
    }
}
```

```
p = mobile();

if (p == -1)
    break;

v = a[p];
t = p + d[p];

int x = a[p];
a[p] = a[t];
a[t] = x;

x = d[p];
d[p] = d[t];
d[t] = x;

for (i = 0; i < n; i++)
{
    if (a[i] > v)
        d[i] = -d[i];
}

return 0;
}
```

```
Enter n: 4
```

```
1 2 3 4
```

```
1 2 4 3
```

```
1 4 2 3
```

```
4 1 2 3
```

```
4 1 3 2
```

```
1 4 3 2
```

```
1 3 4 2
```

```
1 3 2 4
```

```
3 1 2 4
```

```
3 1 4 2
```

```
3 4 1 2
```

```
4 3 1 2
```

```
4 3 2 1
```

```
3 4 2 1
```

```
3 2 4 1
```

```
3 2 1 4
```

```
2 3 1 4
```

```
2 3 4 1
```

```
2 4 3 1
```

```
4 2 3 1
```

```
4 2 1 3
```

```
2 4 1 3
```

```
2 1 4 3
```

```
2 1 3 4
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include<stdio.h>
#include<time.h>

void merge(int* a,int l,int m,int r){
    int c = l, b = m+1, k = 0;
    int v[50];
    while(c <= m && b <= r){
        if(a[c] < a[b]) v[k++] = a[c++];
        else v[k++] = a[b++];
    }
    while(c <= m) v[k++] = a[c++];
    while(b <= r) v[k++] = a[b++];
    for(int i=l,j=0;i<=r;i++,j++) a[i] = v[j];
}

void mergesort(int l,int r,int* a){
    if(l >= r) return;
    int m = (l+r)/2;
    mergesort(l,m,a);
    mergesort(m+1,r,a);
    merge(a,l,m,r);
}

int main(){
    int n;
    int a[100];
    printf("Enter number of elements:");
    scanf("%d",&n);
    for (int i = 0; i < n; i++) {
        scanf("%d",&a[i]);
    }
    clock_t start,end;
    start = clock();
    mergesort(0,n-1,a);
    end = clock(); // end time
    printf("Sorted array:\n");
```

```

for(int i=0;i<n;i++) printf("%d ",a[i]);
double time_taken = ((double)(end-start))/CLOCKS_PER_SEC;
printf("\nTime taken: %f seconds\n",time_taken);
return 0;
}

```

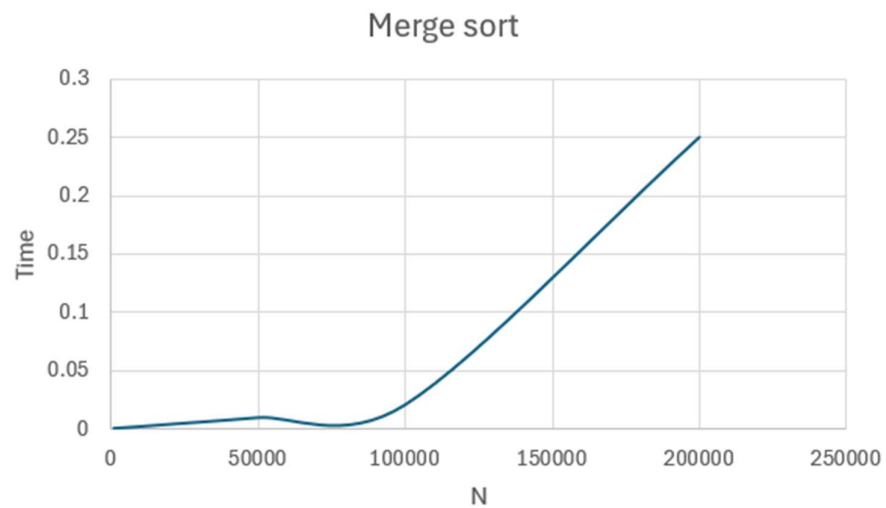
OUTPUT

```

Enter number of elements:9
3 4 1 3 5 6 2 2 1
Sorted array:
1 1 2 2 3 3 4 5 6
Time taken: 0.000017 seconds

...Program finished with exit code 0
Press ENTER to exit console.

```



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>
int partition(int* a, int l,int n){
    int i = l+1,j = n;
    while(1){
        while(i <= n && a[i] <= a[l]) i++;
        while(a[j] > a[l]) j--;
        if(i >= j) break;
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
    int t = a[l];
    a[l] = a[j];
    a[j] = t;
    return j;
}
void quicksort(int* a,int l,int n){
    if(l >= n) return;
    int pivotIdx = partition(a,l,n);
    quicksort(a,0,pivotIdx-1);
    quicksort(a,pivotIdx+1,n);
}
int main()
{
    int n;
    int a[100];
    printf("Enter number of elements:");
    scanf("%d",&n);
    for (int i = 0; i < n; i++) {
        scanf("%d",&a[i]);
    }
    clock_t start,end;
    start = clock();
```

```

quicksort(a,0,n-1);
end = clock();

for (int i = 0; i < n; i++) {
    printf("%d ",a[i]);
}
double time_taken = (((double)(end-start))/CLOCKS_PER_SEC)*1000;
printf("\nTime taken: %f milli seconds\n",time_taken);
}

```

```

Enter number of elements:9
23 343 2 1 3 4 4 5 1
1 1 2 3 4 4 5 23 343
Time taken: 0.002000 milli seconds

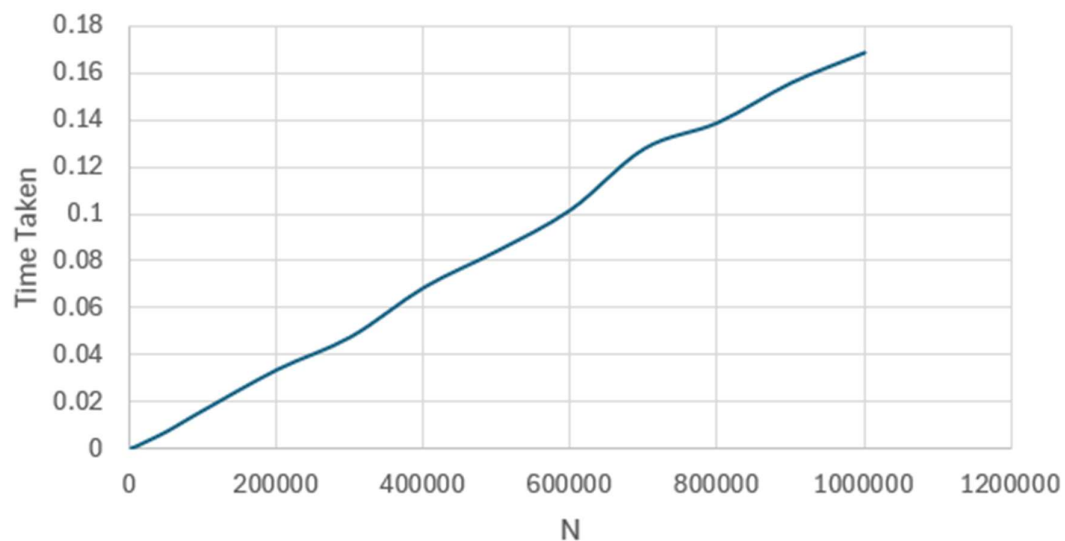
```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Quick sort



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include<stdio.h>
#include<time.h>

void heapify(int* a,int n,int p){
    int c,item;
    c = 2*p+1;
    item = a[p];
    while(c <= n-1){
        if(c+1 <= n-1){
            if(a[c] < a[c+1]) c++;
        }
        if(item < a[c]){
            a[p] = a[c];
            p = c;
            c = 2*p+1;
        }else break;
    }
    a[p] = item;
}

void buildheap(int* a, int n){
    for(int i = (n-1)/2; i >= 0; i--){
        heapify(a,n,i);
    }
}

void heapsort(int* a,int n){
    buildheap(a,n);
    for(int i = n-1; i > 0; i--){
        int t = a[i];
        a[i] = a[0];
        a[0] = t;
        heapify(a,i,0);
    }
}

int main(){
    int n;
    int a[100];
    printf("Enter number of elements:");
    scanf("%d",&n);
    for (int i = 0; i < n; i++) {
        scanf("%d",&a[i]);
    }
    clock_t start,end;
    start = clock();
    heapsort(a,n);
    end = clock();
    printf("Sorted array:\n");
    for(int i=0;i<n;i++)
        printf("%d ",a[i]);
}
```

```

double time_taken = ((double)(end-start))/CLOCKS_PER_SEC;

printf("\nTime taken: %f seconds\n",time_taken);

return 0;
}

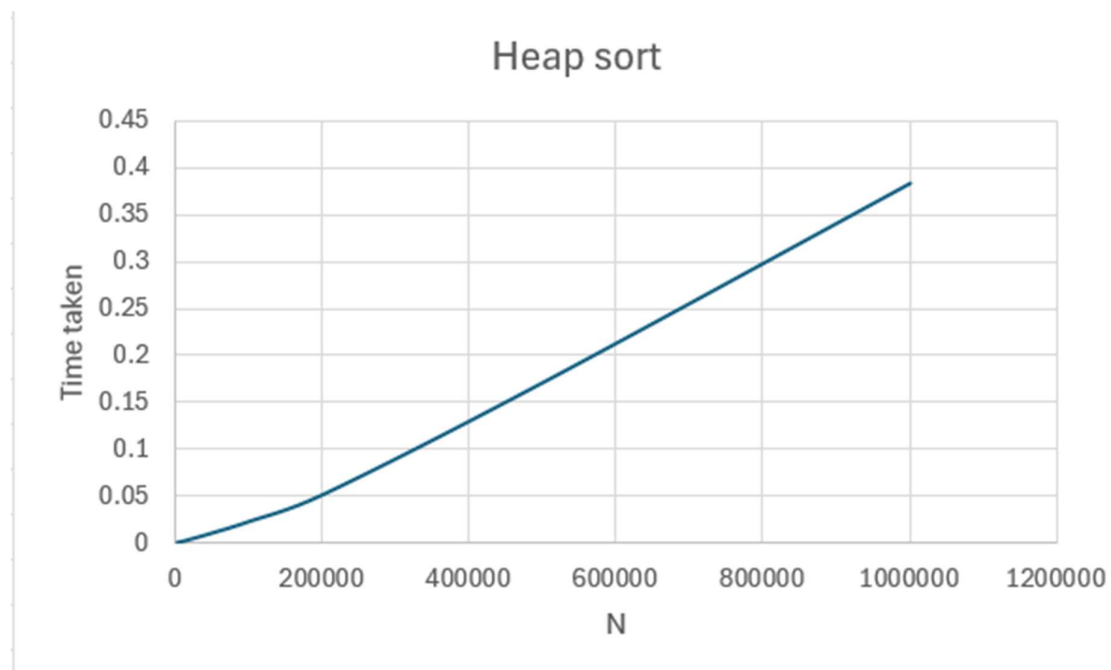
```

```

Enter number of elements:9
3 2 1 3 5 6 7 8 0
Sorted array:
0 1 2 3 3 5 6 7 8
Time taken: 0.000003 seconds

...Program finished with exit code 0
Press ENTER to exit console.

```



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int W, int wt[], int val[], int n) {
    int i, w;
    int K[n + 1][W + 1];

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    return K[n][W];
}

int main() {
    int val[100], wt[100], n, W;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter values of items: ");
    for(int i = 0; i < n; i++) scanf("%d", &val[i]);
```

```
printf("Enter weights of items: ");  
for(int i = 0; i < n; i++) scanf("%d", &wt[i]);  
  
printf("Enter capacity of knapsack: ");  
scanf("%d", &W);  
  
printf("Maximum profit: %d\n", knapsack(W, wt, val, n));  
return 0;  
}
```

```
Enter number of items: 3  
Enter values of items: 12 33 22  
Enter weights of items: 2 3 4  
Enter capacity of knapsack: 5  
Maximum profit: 45  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include<stdio.h>

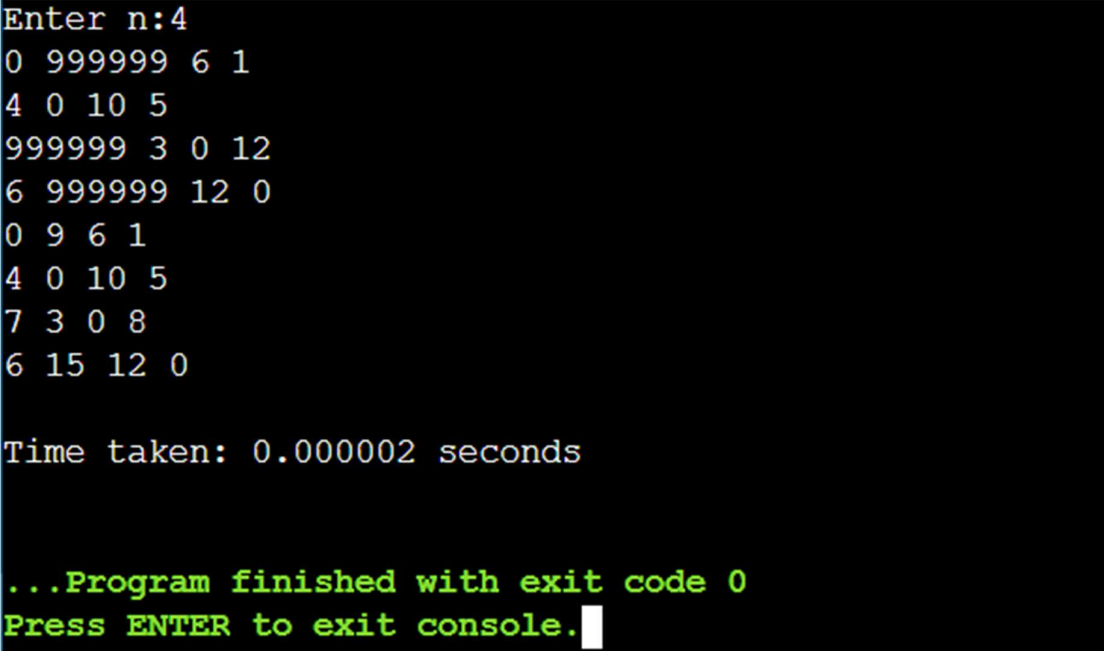
#include<time.h>

int min(int a,int b){
    return a < b ? a : b;
}

void floyd(int a[][100],int n){
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            for(int k=0;k<n;k++){
                a[j][k] = min(a[j][k],a[j][i]+a[i][k]);
            }
        }
    }
}

int main(){
    int n;
    int a[100][100];
    printf("Enter n:");
    scanf("%d",&n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d",&a[i][j]);
        }
    }
    clock_t start,end;
    start = clock();
    floyd(a,n);
    end = clock();
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++)printf("%d ",a[i][j]);
```

```
    printf("\n");  
}  
  
double time_taken = ((double)(end-start))/CLOCKS_PER_SEC;  
  
printf("\nTime taken: %f seconds\n",time_taken);  
  
return 0;  
}
```



```
Enter n:4  
0 999999 6 1  
4 0 10 5  
999999 3 0 12  
6 999999 12 0  
0 9 6 1  
4 0 10 5  
7 3 0 8  
6 15 12 0  
  
Time taken: 0.000002 seconds  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Lab program 8:

(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include<stdio.h>
#include<time.h>
struct edge {
    int s, e, cost;
};
int n, cost = 0, nv;
int vis[100];
struct edge a[100];
int main() {
    if (scanf("%d %d", &n, &nv) != 2) return 0;
    printf("Enter s, e and cost for following:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &a[i].s, &a[i].e, &a[i].cost);
    }
    clock_t start,end;

    start = clock();
    vis[a[0].s] = 1;
    int edges_count = 0;

    while (edges_count < nv - 1) {
        int min = 100000;
        int minIdx = -1;

        for (int i = 0; i < n; i++) {
            if ((vis[a[i].s] && !vis[a[i].e]) || (!vis[a[i].s] && vis[a[i].e))) {
                if (a[i].cost < min) {
                    min = a[i].cost;
                    minIdx = i;
                }
            }
        }
    }
```

```

    }

    if (minIdx != -1) {
        cost += min;
        vis[a[minIdx].s] = 1;
        vis[a[minIdx].e] = 1;
        edges_count++;
    } else {
        break;
    }
}

end = clock();
printf("The total cost is: %d\n", cost);
double time_taken = ((double)(end-start))/CLOCKS_PER_SEC;

printf("\nTime taken: %f seconds\n",time_taken);
return 0;
}

```

```

9
10
Enter s, e and cost for following:
1 2 3
2 3 7
2 5 6
5 6 5
6 4 1
6 9 11
8 9 10
10 9 4
7 10 2
The total cost is: 49

Time taken: 0.000003 seconds

```

(b)Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include<stdio.h>
#include<time.h>
struct edge{
    int s,e,cost;
};
int cost = 0;
int grp[100];
int find(int x){
    if(x != grp[x]) grp[x] = find(grp[x]);
    return grp[x];
}
void merge(int x,int y,int c){
    int gx = find(x);
    int gy = find(y);
    if(gx != gy){
        grp[gy] = gx;
        cost += c;
    }
}
int main(){
    struct edge a[100];
    int n;
    scanf("%d",&n);
    printf("Enter s,e and cost for following");
    for(int i = 0; i < n; i++){
        scanf("%d",&a[i].s);
        scanf("%d",&a[i].e);
        scanf("%d",&a[i].cost);
    }
    clock_t start,end;

    start = clock();
```

```

for(int i = 0; i < n; i++){
    for(int j = i+1; j < n; j++){
        if(a[i].cost > a[j].cost){
            struct edge t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }
}

for(int i = 0; i < 100; i++) grp[i]=i;
for(int i = 0; i < n; i++){
    merge(a[i].s,a[i].e,a[i].cost);
}

end = clock();

printf("The total cost is: %d",cost);

double time_taken = ((double)(end-start))/CLOCKS_PER_SEC;

printf("\nTime taken: %f seconds\n",time_taken);

return 0;
}

```

```

9
Enter s,e and cost for following1 2 3
2 3 7
2 5 6
5 6 5
6 4 1
6 9 11
8 9 10
10 9 4
7 10 2
The total cost is: 49
Time taken: 0.000004 seconds

...Program finished with exit code 0
Press ENTER to exit console.

```

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

```
#include <stdio.h>

void swap(float *a, float *b)
{
    float t = *a;
    *a = *b;
    *b = t;
}

int main()
{
    int n;

    printf("Enter no of elements, prices and weights:\n");
    scanf("%d", &n);

    float p[100], wt[100], temp[100];

    for (int i = 0; i < n; i++)
        scanf("%f", &p[i]);

    for (int i = 0; i < n; i++)
    {
        scanf("%f", &wt[i]);

        if (wt[i] == 0)
            return 0;

        temp[i] = p[i] / wt[i];
    }

    /* Sort in descending order of profit/weight ratio */
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (temp[i] < temp[j])
            {
                swap(&p[i], &p[j]);
                swap(&temp[i], &temp[j]);
                swap(&wt[i], &wt[j]);
            }
        }
    }

    int m;
    float res = 0;

    printf("Enter capacity: ");
    scanf("%d", &m);

    for (int i = 0; i < n; i++)
```

```

{
    if (m <= 0)
        break;

    float x;

    if (m < wt[i])
        x = m;
    else
        x = wt[i];

    res += temp[i] * x;
    m -= x;
}

printf("The maximum profit is %.2f\n", res);

return 0;
}

```

```

Enter no of elements, prices and weights:
3
1 2 1
2 1 2
Enter capacity: 2
The maximum profit is 2.50

...Program finished with exit code 0
Press ENTER to exit console.

```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>
void swap(float* a,float* b){
    float t = *a;
    *a = *b;
    *b = t;
}
void main(){
    int n;
    printf("Enter no of vertices");
    scanf("%d",&n);
    int dist[100], adj[2][100][100],vis[100];
    for(int i = 0; i < n; i++){
        dist[i] = 10000;
        vis[i] = 0;
        for(int j = 0; j < n; j++){
            scanf("%d",&adj[0][i][j]);
            scanf("%d",&adj[1][i][j]);
        }
    }
    dist[0] = 0;
    for(int s = 0; s < n; s++){
        int curr = 10001,idx = 0;
        for(int j = 0; j < n; j++){
            if(!vis[j] && curr > dist[j]){
                curr = dist[j];
                idx = j;
            }
        }
        for(int i = 0; i < n; i++){
            if(!vis[i] && adj[0][idx][i] && dist[i] > dist[idx]+adj[1][idx][i]){
                dist[i] = dist[idx]+adj[1][idx][i];
            }
        }
        vis[idx] = 1;
    }
    for(int i = 0; i < n; i++){
        printf("The min distance is for %d: %d \n",i,dist[i]);
    }
}
```

```
Enter no of vertices4
0 0 1 1 0 1 2 0
0 0 0 0 0 0 0 0
2 1 0 0 1 2 0 0
0 0 1 2 0 0 0 0
The min distance is for 0: 0
The min distance is for 1: 1
The min distance is for 2: 10000
The min distance is for 3: 0

...Program finished with exit code 4
Press ENTER to exit console.
```

Lab program 10:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <time.h>
#include <math.h>

int mat[100];

int checkValid(int row, int col) {
    for(int i = 0; i < row; i++) {
        if(mat[i] == col || abs(i - row) == abs(mat[i] - col))
            return 0;
    }
    return 1;
}

void nqueens(int row, int n) {
    if(row == n) {
        printf("Solution:\n");

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(mat[i] == j)
                    printf("Q ");
                else
                    printf(". ");
            }
            printf("\n");
        }

        printf("\n");
        return;
    }

    for(int col = 0; col < n; col++) {
        if(checkValid(row, col)) {
            mat[row] = col;
            nqueens(row + 1, n);
        }
    }
}

int main() {
    int n = 4;
```

```

clock_t start, end;
start = clock();

nqueens(0, n);

end = clock();

double time_taken =
    ((double)(end - start) / CLOCKS_PER_SEC) * 1000;

printf("Time taken: %f milliseconds\n", time_taken);

return 0;
}

```

Solution:

```

. Q . .
. . . Q
Q . . .
. . Q .

```

Solution:

```

. . Q .
Q . . .
. . . Q
. Q . .

```

Time taken: 0.055000 milliseconds

...Program finished with exit code 0
Press ENTER to exit console.

11. Container With Most Water

You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the i^{th} line are (i, 0) and (i, height[i]).

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

```
#include <stdio.h>

static inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}

static inline int min_int(int a, int b) {
    return (a < b) ? a : b;
}

int maxArea(int* height, int heightSize) {
    int l = 0;
    int r = heightSize - 1;
    int res = -1;
    while (l < r) {
        int current_height = min_int(height[l], height[r]);
        int current_width = r - l;
        int area = current_height * current_width;
        res = max_int(res, area);
        if (height[l] < height[r]) {
            l++;
        } else {
            r--;
        }
    }
    return res;
}
```

}

Code

Accepted

×

← All Submissions

Accepted

65 / 65 testcases passed

ChetanBarakki submitted at Feb 23, 2026 14:27

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

62.82 MB | Beats 77.70%

Time Interval	Performance (%)
0ms	~55%
1ms	~5%
2ms	~5%
3ms	~10%
4ms	~10%
5ms	~5%
6ms	~5%

Code | C++

```
1 class Solution {
2 public:
3     int maxArea(vector<int>& height) {
4         int n = height.size();
5         int l = 0, r = n-1;
6         int res = -1;
7         while(l < r){
8             res = max(res, min(height[l], height[r])*(r-l));
```

Testcase

Test Result

34. Find First and Last Position of Element in Sorted Array

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with $O(\log n)$ runtime complexity.

```
#include <stdio.h>

#include <stdlib.h>

int* searchRange(int* nums, int numsSize, int target, int* returnSize) {

    *returnSize = 2;

    int* result = (int*)malloc(2 * sizeof(int));

    result[0] = -1;

    result[1] = -1

    if (numsSize == 0) return result;

    int l = 0, r = numsSize - 1, idx = -1;

    while (l <= r) {

        int mid = l + (r - l) / 2;

        if (nums[mid] == target) {

            idx = mid;

            break;

        } else if (nums[mid] > target) {

            r = mid - 1;

        } else {

            l = mid + 1;

        }

    }

    if (idx == -1) return result;

    int i = idx, j = idx;

    while (i + 1 < numsSize && nums[i + 1] == target) i++;

    while (j - 1 >= 0 && nums[j - 1] == target) j--;
```

```

result[0] = j;
result[1] = i;
return result;
}

```

Code

Accepted

All Submissions

Accepted

88 / 88 testcases passed

ChetanBarakki submitted at Feb 23, 2026 14:54

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

17.50 MB | Beats 53.21%

97.42% of solutions used 0 ms of runtime

Code

C++

```

1 class Solution {
2 public:
3     vector<int> searchRange(vector<int>& nums, int target) {
4         int lb = int(lower_bound(begin(nums), end(nums), target)-begin(nums));
5         int ub = int(upper_bound(begin(nums), end(nums), target)-begin(nums)-1);
6         if(lb>ub) return {-1,-1};
7         return {lb,ub};
8     }

```

Testcase

Test Result

268. Missing Number

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array*.

```
int missingNumber(int* nums, int numsSize) {  
    int sum = numsSize * (numsSize + 1) / 2;  
    for (int i = 0; i < numsSize; i++) {  
        sum -= nums[i];  
    }  
    return sum;  
}
```

[← All Submissions](#)

Accepted 122 / 122 testcases passed

ChetanBarakki submitted at Mar 02, 2026 14:10

[Analysis](#)

[Solution](#)

⌚ Runtime

0 ms | Beats 100.00%

💾 Memory

21.85 MB | Beats 37.79%

Code | C++

```
1 class Solution {  
2 public:  
3     int missingNumber(vector<int>& nums) {  
4         int n = nums.size();  
5         int sum = n*(n+1)/2;  
6         for(int i = 0; i < n; i++){  
7             sum -=nums[i];  
8         }  
9     }  
10 }
```

☒ Testcase

[Test Result](#)

1636. Sort Array by Increasing Frequency

Given an array of integers `nums`, sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.

Return the *sorted array*.

```
#include <stdlib.h>

typedef struct {
    int val;
    int freq;
} Element;

int compare(const void* a, const void* b) {
    Element* e1 = (Element*)a;
    Element* e2 = (Element*)b;

    if (e1->freq != e2->freq) {
        return e1->freq - e2->freq;
    }
    return e2->val - e1->val;
}

int* frequencySort(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    if (numsSize == 0) return nums;

    int temp_arr[numsSize];
    for (int i = 0; i < numsSize; i++) temp_arr[i] = nums[i];

    // Simple sort to group identical elements
    for (int i = 0; i < numsSize - 1; i++) {
        for (int j = 0; j < numsSize - i - 1; j++) {
            if (temp_arr[j] > temp_arr[j + 1]) {
                int temp = temp_arr[j];
                temp_arr[j] = temp_arr[j + 1];
                temp_arr[j + 1] = temp;
            }
        }
    }

    Element elements[numsSize];
    int count = 0;
    for (int i = 0; i < numsSize; ) {
        int j = i;
        while (j < numsSize && temp_arr[j] == temp_arr[i]) j++;
        elements[count].val = temp_arr[i];
        elements[count].freq = j - i;
        count++;
        i = j;
    }

    qsort(elements, count, sizeof(Element), compare);
}
```

```

int k = 0;
for (int i = 0; i < count; i++) {
    for (int f = 0; f < elements[i].freq; f++) {
        nums[k++] = elements[i].val;
    }
}

return nums;
}

```

← All Submissions



Accepted 180 / 180 testcases passed

ChetanBarakki submitted at Mar 02, 2026 14:50

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

15.12 MB | Beats 47.60%



Code | C++

```

1 class Solution {
2 public:
3     vector<int> frequencySort(vector<int>& nums) {
4         unordered_map<int,int> f;
5         for(int ele : nums){
6             f[ele]++;
7         }
8         vector<pair<int,int>> v(begin(f),end(f));

```

207. Course Schedule

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

```
#include <stdbool.h>
#include <stdlib.h>
// Simple adjacency list node
typedef struct Node {
    int dest;
    struct Node* next;
} Node;

bool isCyclic(int src, Node** adj, bool* vis, bool* recPath) {
    vis[src] = true;
    recPath[src] = true;
    Node* temp = adj[src];
    while (temp != NULL) {
        int v = temp->dest;
        if (!vis[v]) {
            if (isCyclic(v, adj, vis, recPath)) return true;
        } else if (recPath[v]) {
            return true;
        }
        temp = temp->next;
    }
    recPath[src] = false;
    return false;
}

bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    // Build Adjacency List
    Node** adj = (Node**)calloc(numCourses, sizeof(Node*));
    for (int i = 0; i < prerequisitesSize; i++) {
        int u = prerequisites[i][1];
        int v = prerequisites[i][0];
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->dest = v;
        newNode->next = adj[u];
        adj[u] = newNode;
    }

    bool* vis = (bool*)calloc(numCourses, sizeof(bool));
    bool* recPath = (bool*)calloc(numCourses, sizeof(bool));

    bool possible = true;
    for (int i = 0; i < numCourses; i++) {
        if (!vis[i]) {
```

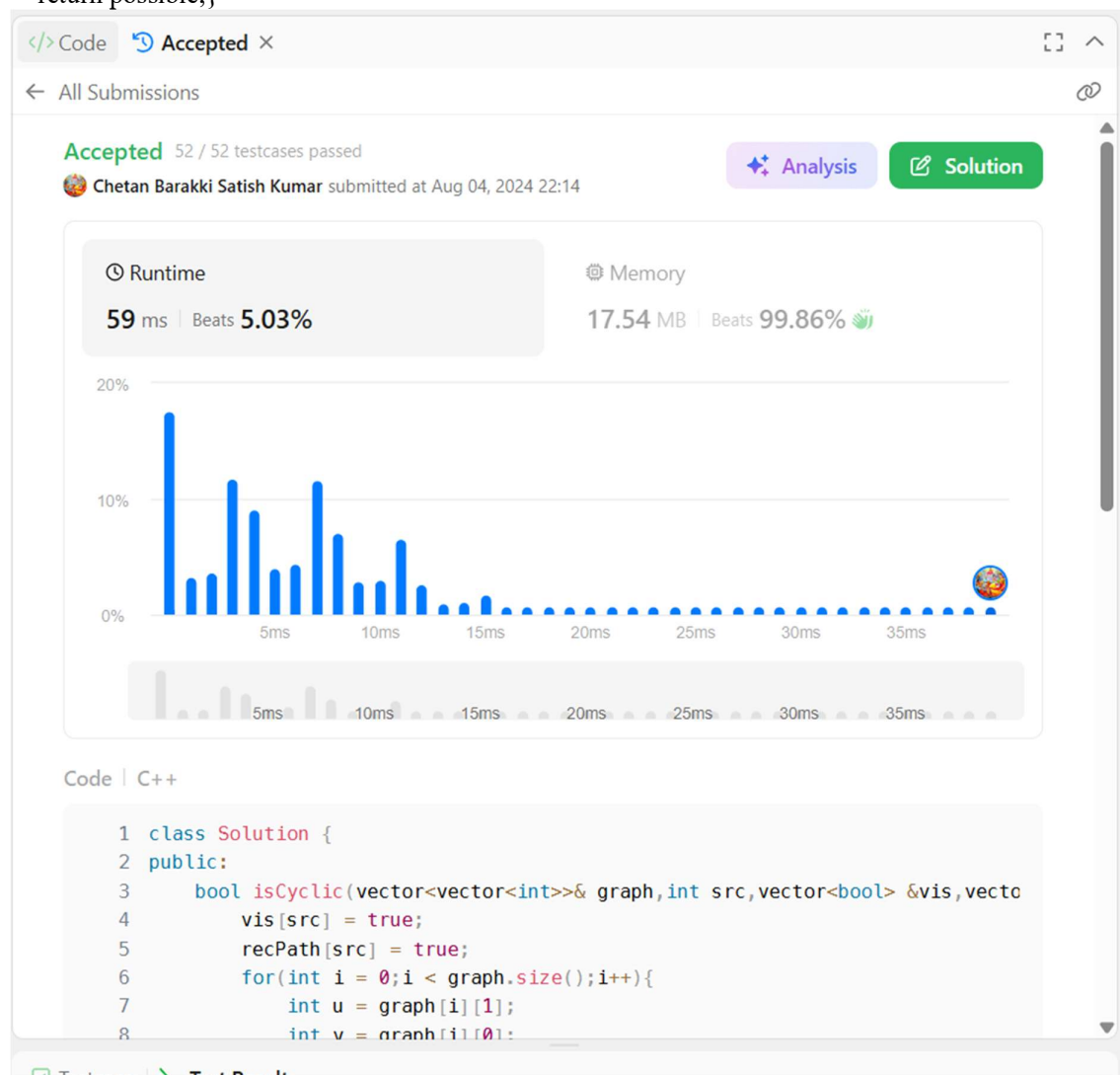
```

        if (isCyclic(i, adj, vis, recPath)) {
            possible = false;
            break;
        }
    }
}

// Cleanup memory
for (int i = 0; i < numCourses; i++) {
    Node* curr = adj[i];
    while (curr) {
        Node* temp = curr;
        curr = curr->next;
        free(temp);
    }
}
free(adj);
free(vis);
free(recPath);

return possible;}

```



287. Find the Duplicate Number

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return *this repeated number*.

You must solve the problem without modifying the array `nums` and using only constant extra space.

```
int findDuplicate(int* nums, int numsSize) {  
    int slow = nums[0];  
    int fast = nums[0];  
  
    while (1) {  
        slow = nums[slow];  
        fast = nums[nums[fast]];  
        if (slow == fast) break;  
    }  
  
    slow = nums[0];  
    while (slow != fast) {  
        slow = nums[slow];  
        fast = nums[fast];  
    }  
  
    return slow;  
}
```

</> Code Accepted ×

← All Submissions 🔗

Accepted 59 / 59 testcases passed

👤 ChetanBarakki submitted at May 11, 2026 15:26

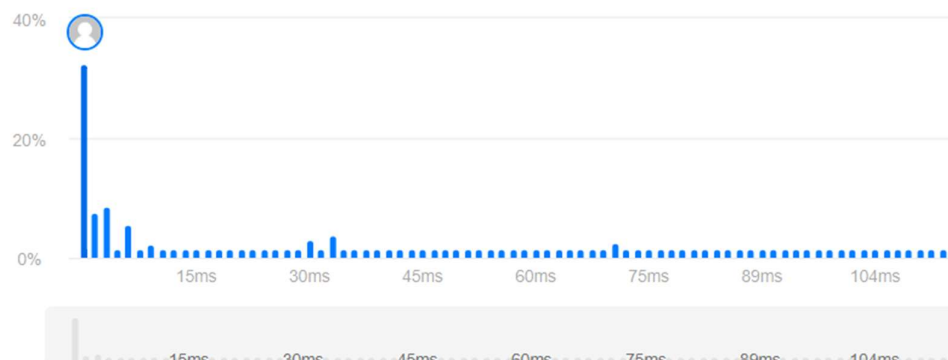
🌟 Analysis 📄 Solution

🕒 Runtime

0 ms | Beats 100.00% 🌿

💾 Memory

65.06 MB | Beats 69.47% 🌿



Code | C++

```
1 class Solution {
2 public:
3     int findDuplicate(vector<int>& nums) {
4         int slow = nums[0], fast = nums[0];
5         while(true){
6             slow = nums[slow];
7             fast = nums[nums[fast]];
8         }
9     }
10 }
```

801. Minimum Swaps To Make Sequences Increasing

You are given two integer arrays of the same length `nums1` and `nums2`. In one operation, you are allowed to swap `nums1[i]` with `nums2[i]`.

- For example, if `nums1 = [1,2,3,8]`, and `nums2 = [5,6,7,4]`, you can swap the element at `i = 3` to obtain `nums1 = [1,2,3,4]` and `nums2 = [5,6,7,8]`.

Return the minimum number of needed operations to make `nums1` and `nums2` strictly increasing. The test cases are generated so that the given input always makes it possible.

An array `arr` is strictly increasing if and only if `arr[0] < arr[1] < arr[2] < ... < arr[arr.length - 1]`.

```
#define MIN(a, b) (a <= b? a : b)
#define MAX(a, b) (a >= b? a : b)

int minSwap(int* A, int ASize, int* B, int BSize){
    int ans = 0;
    int e = 0; // the max. good index before the current iteration i
    int curSwaps = 0; // record no. of swaps in the current subproblem
    for (int i = 1; i < ASize; i++) {
        if (MIN(A[i], B[i]) > MAX(A[i-1], B[i-1])) { // if i is a good index
            ans += MIN(curSwaps, i-e-curSwaps); // select min. swaps of the two configurations
            e = i;
            curSwaps = 0;
            continue;
        }
        if (A[i-1] >= A[i] || B[i-1] >= B[i]) { // if need to swap i
            int temp = A[i];
            A[i] = B[i];
            B[i] = temp;
            curSwaps++;
        }
    }
    ans += MIN(curSwaps, ASize-e-curSwaps);
    return ans;
}
```

Code | Accepted

All Submissions

Accepted 117 / 117 testcases passed

ChetanBarakki submitted at Apr 27, 2026 15:27

Analysis

Solution

Runtime

0 ms | Beats 100.00%

Memory

16.43 MB | Beats 46.88%

Time Interval	Percentage
0-1ms	~35%
1-2ms	~10%
2-3ms	~8%
3-4ms	~15%
4-5ms	~14%
5-6ms	~5%
6-7ms	~3%
7-8ms	~5%

Code | C

```
1 #define MIN(a, b) (a <= b? a : b)
2 #define MAX(a, b) (a >= b? a : b)
3
4 int minSwap(int* A, int ASize, int* B, int BSize){
5     int ans = 0;
6     int e = 0; // the max. good index before the current iteration i
7     int curSwaps = 0; // record no. of swaps in the current subproblem
8     for (int i = 1; i < ASize; i++) {
```

Testcase

Test Result

You must run your code first